Building RAG for WordPress

A deep dive into the Posts RAG Manager Plugin: From Custom Tables to Vector Search.

An architectural deconstruction for developers and students.

Bridging CMS with AI Context

WordPress excels at managing content. RAG (Retrieval-Augmented Generation) requires managing context. The Posts RAG Manager bridges these systems.

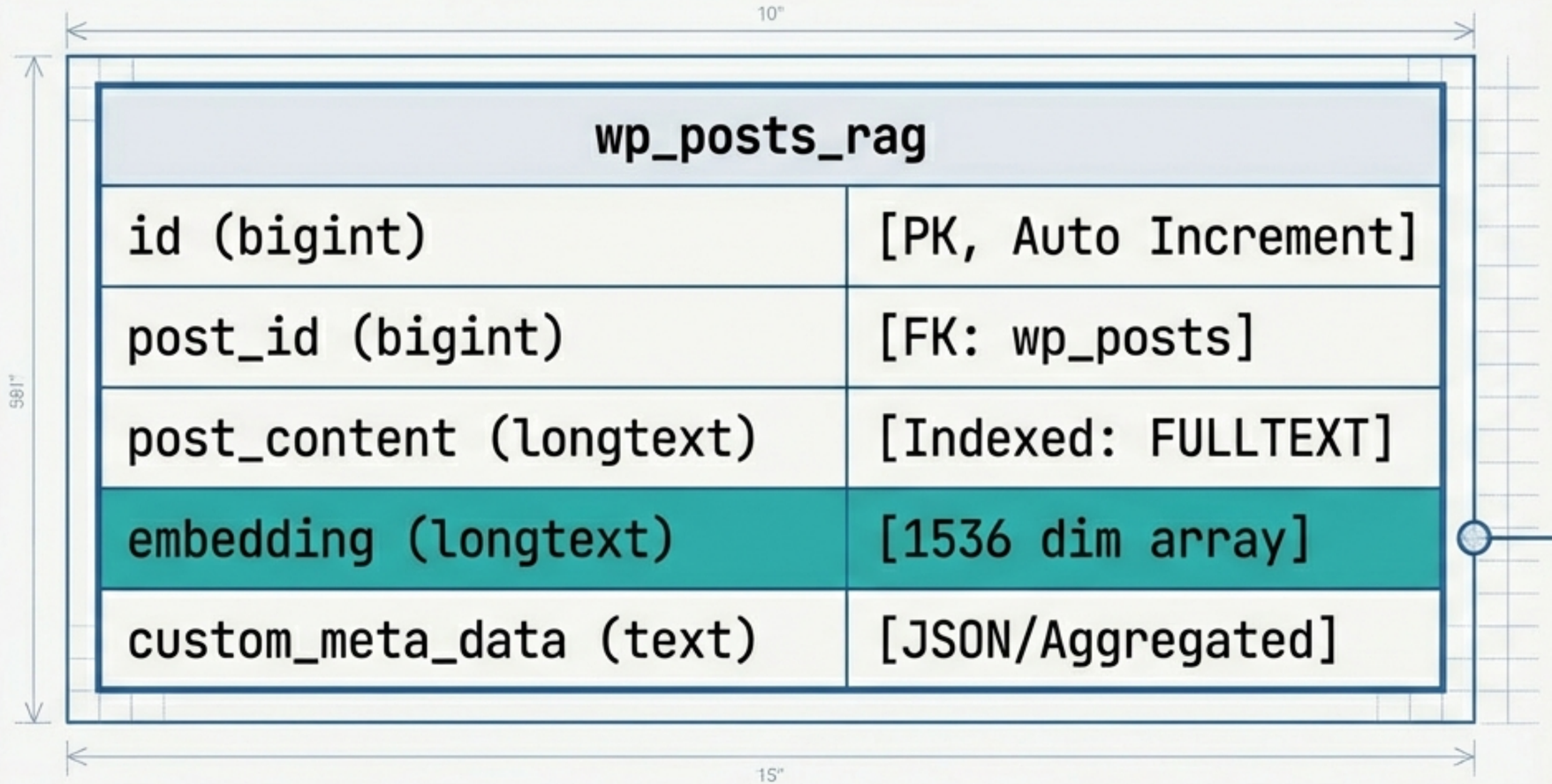
The Problem: Standard WordPress tables are not optimized for heavy vector data.

The Solution:

- Performance: Dedicated tables.
- Isolation: Protecting core WP integrity.
- Intelligence: Vectors alongside text.

Engineered for Performance: The wp_posts_rag Table

Bypassing standard schema limitations for speed.

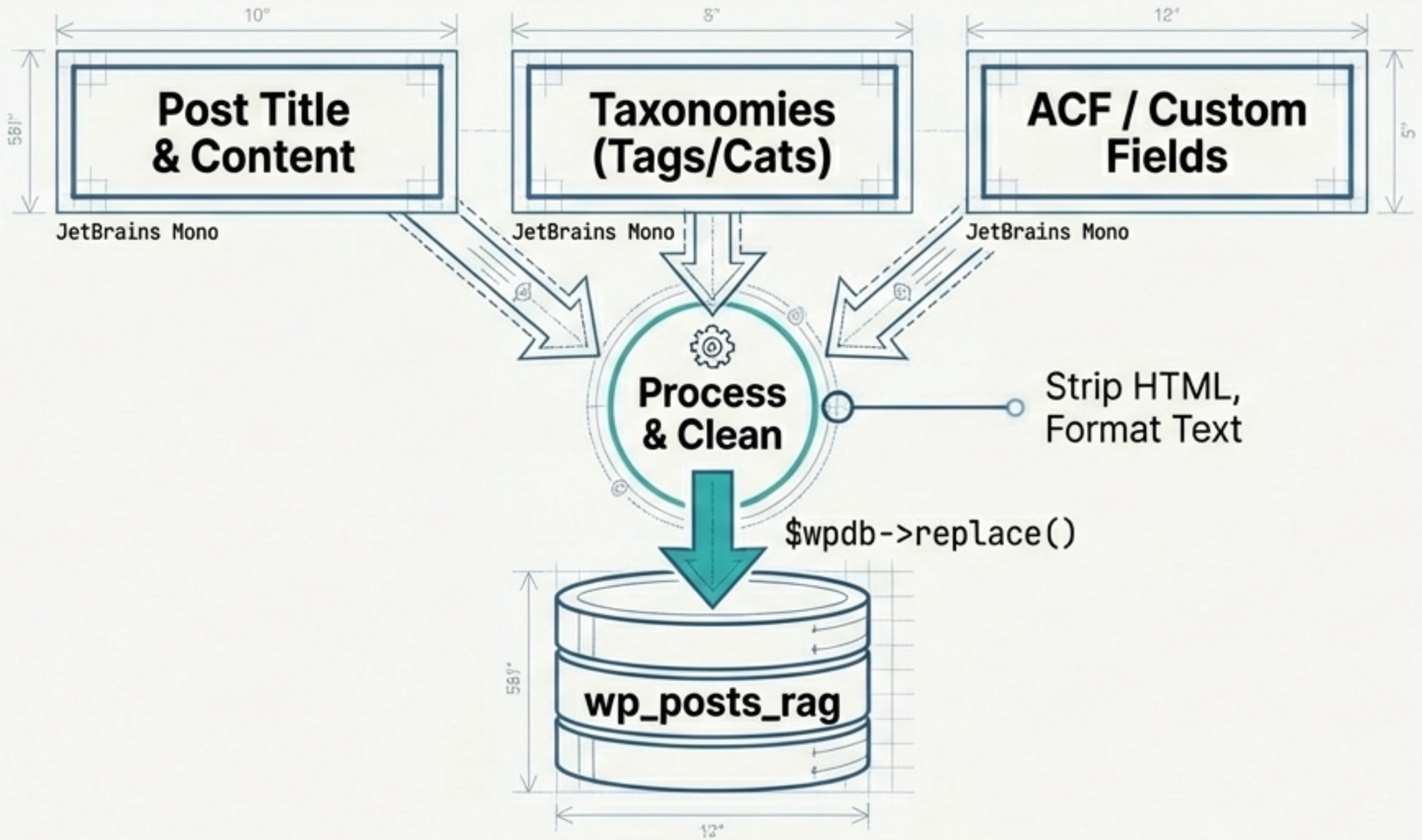


wp_posts_rag	
id (bigint)	[PK, Auto Increment]
post_id (bigint)	[FK: wp_posts]
post_content (longtext)	[Indexed: FULLTEXT]
embedding (longtext)	[1536 dim array]
custom_meta_data (text)	[JSON/Aggregated]

Key Insight: Keeping heavy vector payloads separate prevents slowing down standard site queries.

Moving Data: The Synchronization Engine

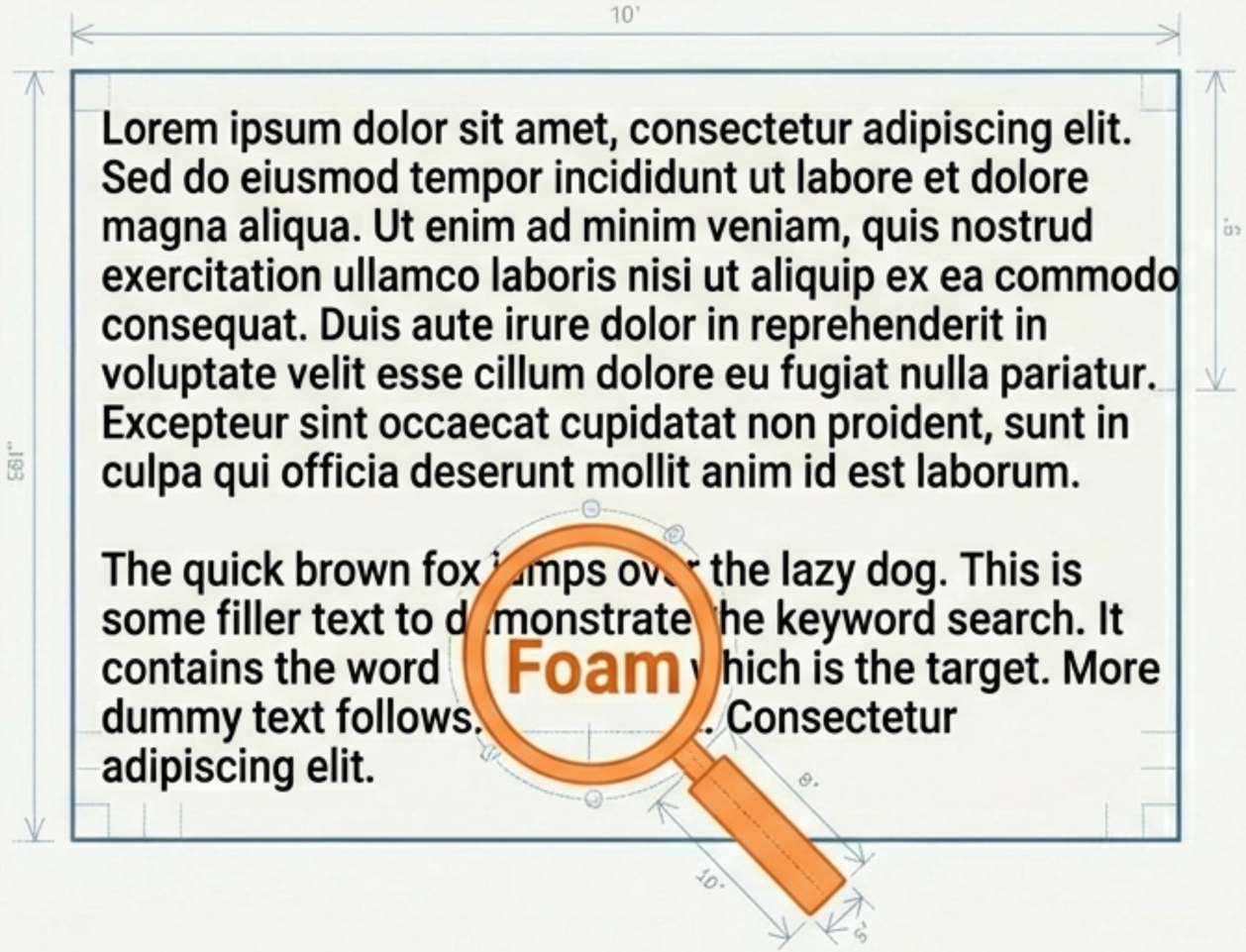
The `sync_posts_to_table()` ETL Pipeline



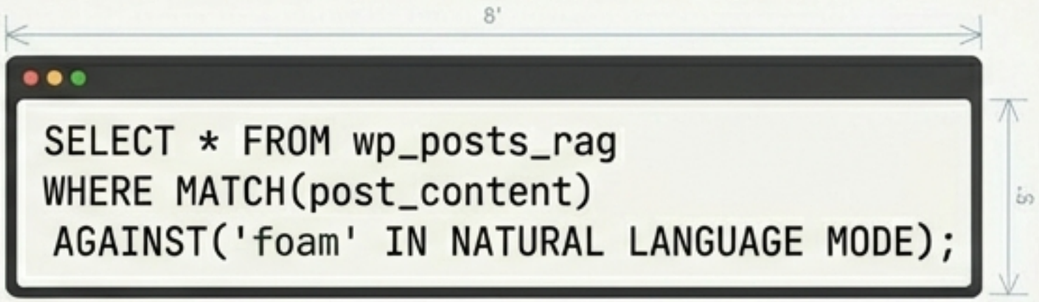
1. **Fetch:** Queries published posts.
2. **Extract:** Pulls metadata & ACF.
3. **Insert:** Populates the staging table.

Engine 1: Full-Text Search (FTS)

The Classic Approach: Keyword Precision



Mechanism: MySQL FULLTEXT index tokenizes text for exact string matches.



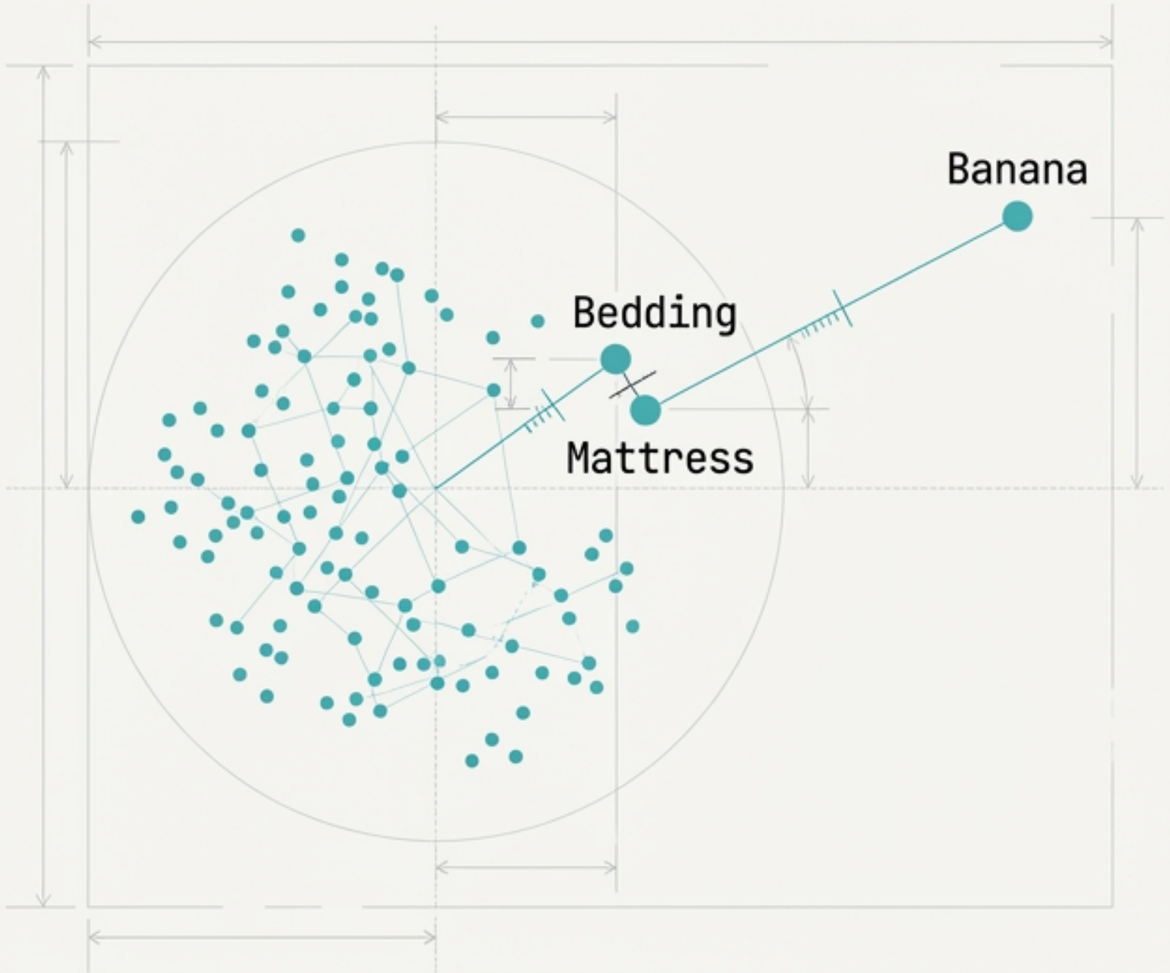
JetBrains Mono

Use Case: Precise terminology.

- Fast Execution
- SQL Native
- User knows the exact keyword.

Engine 2: Vector Search

The Modern Approach: Semantic Understanding



Mechanism: Matches meaning using OpenAI Embeddings.

The Data:
Text is converted into a 1,536-dimensional array of float numbers.

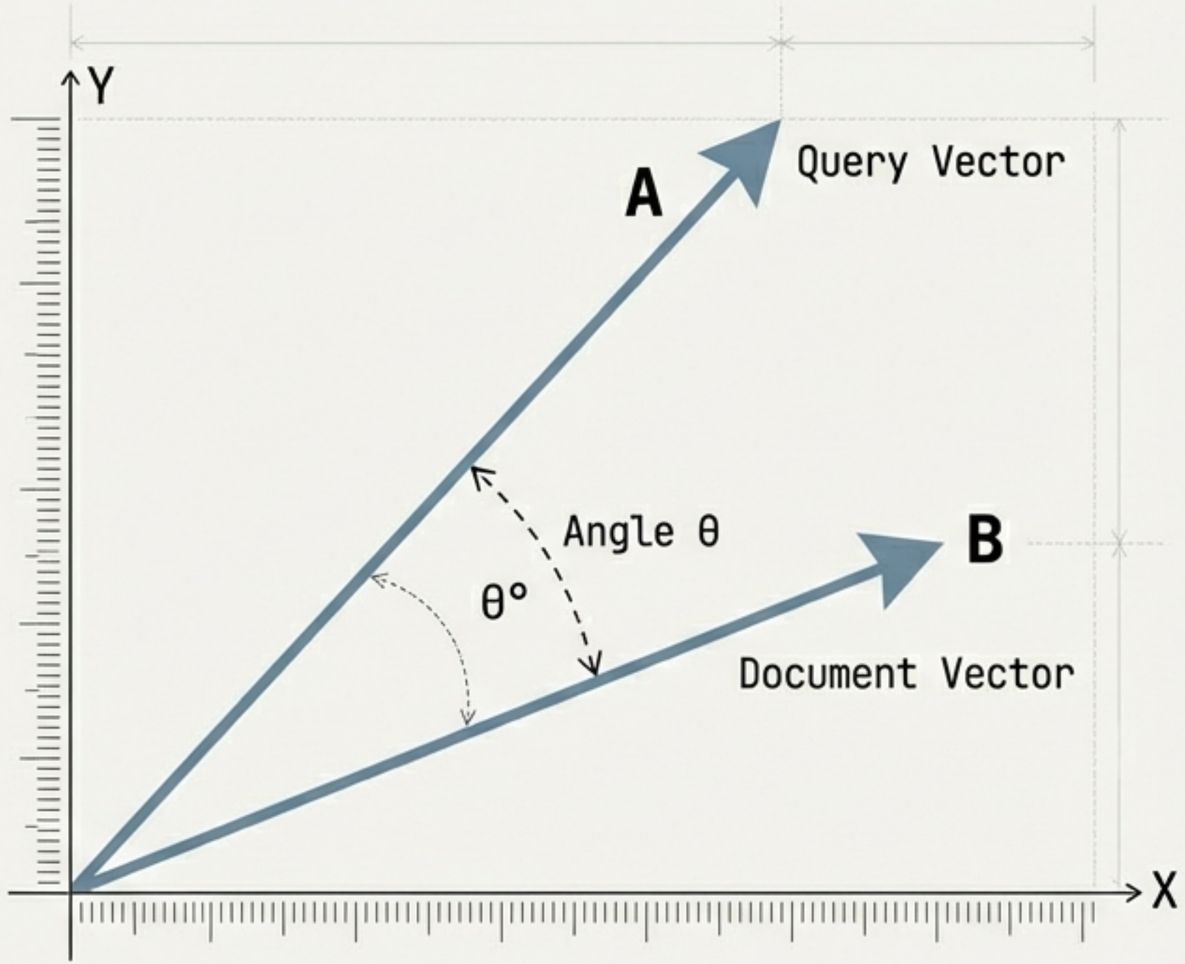
The Logic:

- 'King' - 'Man' + 'Woman' ≈ 'Queen'
- Concepts share numerical proximity.

Example:
Searching 'Comfortable bedding' finds 'Foam Mattress' (No shared keywords).

How Machines Measure Meaning

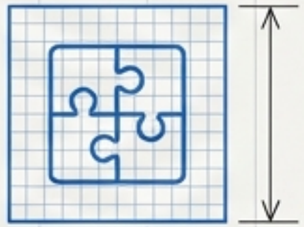
Cosine Similarity: It is geometry, not magic.



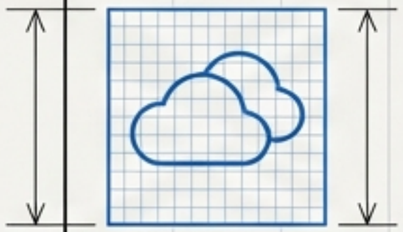
$$\text{Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

- 1.0 = Identical (0° Angle)
- 0.0 = Unrelated (90° Angle)
- -1.0 = Opposite (180° Angle)

Choosing the Right Tool



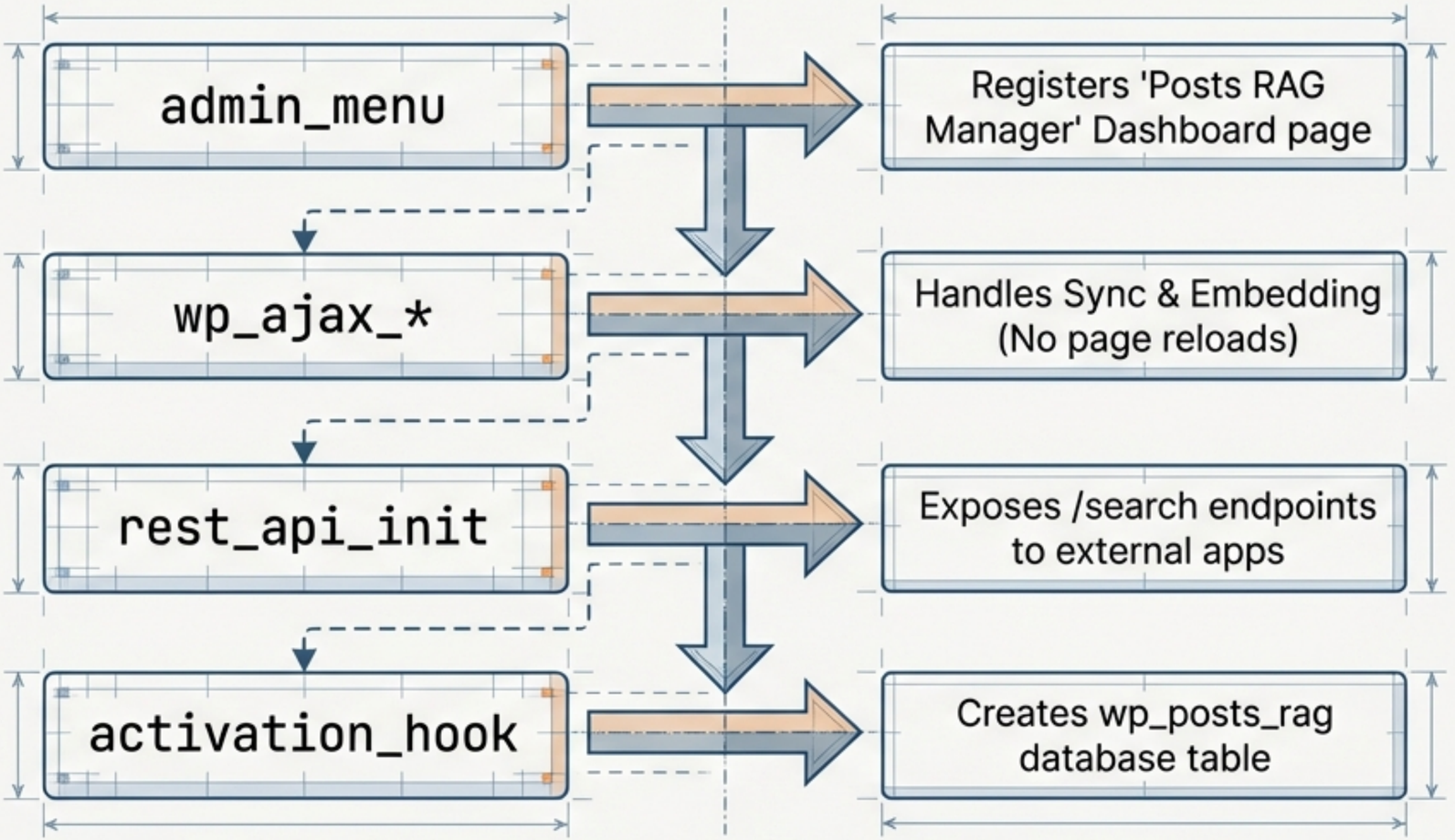
Full-Text Search (FTS)



Vector Search

Full-Text Search (FTS)	Vector Search
Method: Keyword Matching	**Method:** Semantic Similarity
Data: Original Text	**Data:** Numerical Embeddings
Speed: Very Fast (SQL Index)	**Speed:** Slower (API + Math)
Match Type: Literal	**Match Type:** Conceptual
Example: Contains "foam" OR "mattress"	**Example:** Concepts similar to "soft bed"

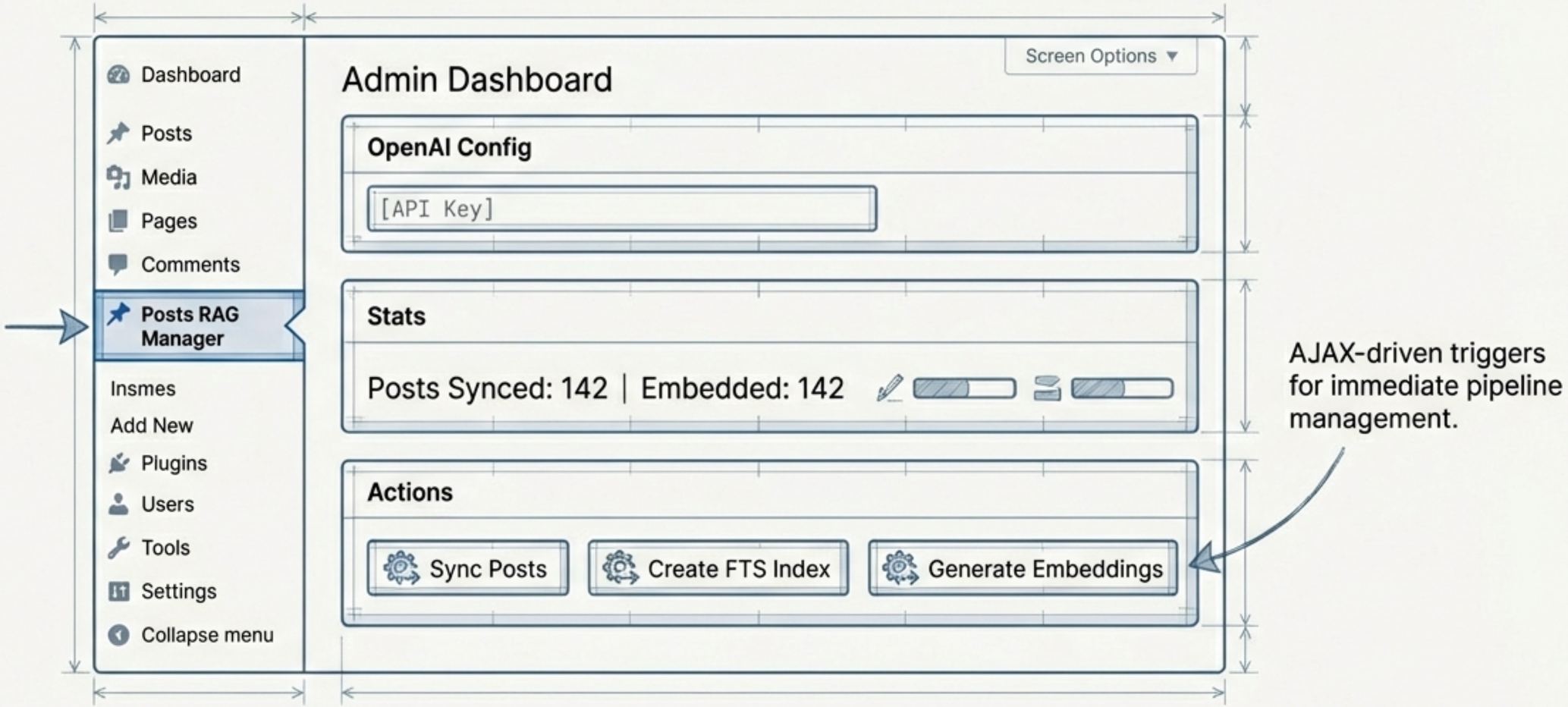
Under the Hood: Plugin Architecture



Lifecycle: Activation -> Sync -> Embed -> Search

The Admin Interface

The Control Center



Headless Access via REST API

Exposing search logic to the world.

Endpoint Details

Endpoint:	/wp-json/posts-rag/v1/search
Type:	Full-Text Keyword
Params:	?query=term&limit=5

Endpoint Details

Endpoint:	/wp-json/posts-rag/v1/vector-search
Type:	Semantic Vector
Params:	?query=term&limit=5

JSON Response Example

```
{
  'id': 101,
  'title': 'Memory Foam Benefits',
  'score': 0.892,
  'link': '...'
}
```

Debugging the RAG Pipeline

Common friction points and fixes.

	Issue	Fix
	Index not created	**Manual Action:** Click ‘Create Index’ in Admin Dashboard.
	No embeddings found	**Pipeline Break:** Run ‘Generate Embeddings’ to populate vectors.
	API Errors / 401	**Credential Error:** Verify OpenAI Key is valid and has credits.

Level Up: Extend the Plugin

Student Exercises

Boolean Logic

Modify FTS SQL to support '+' (include) and '-' (exclude) operators.

Hybrid Search

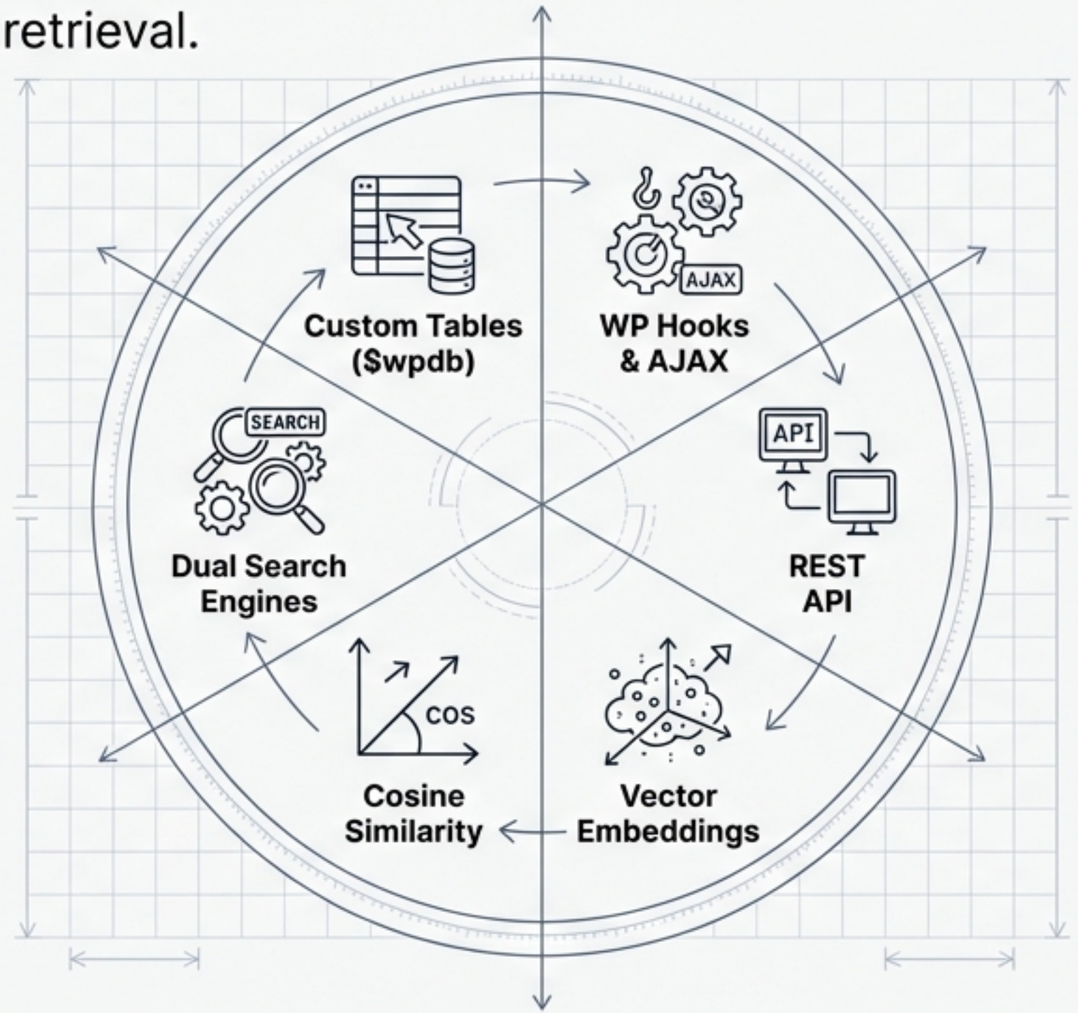
Write a function to normalize and combine FTS scores with Vector Similarity Similarity scores.

UI Feedback

Implement a real-time progress bar for the embedding generation loop using JS.

The RAG Manager Ecosystem

The blueprint for intelligent retrieval.



You now possess the foundation for your next intelligent document retrieval system.